Framework for System Design interviews

Step 1: Deciding requirements

1. **Functional**
2. **Non-Functional**

Functional Requirement: What should the system do?

For eg. If you're designing the YouTube – main functional req. is user can upload videos and user can watch videos

Non-Functional requirement – How well the system can perform. Eg. Uploading a video shouldn't take hrs. latency matters! Streaming videos shouldn't pause every 10 seconds. System should handle massive traffic.

Step 2. Capacity Estimation

We will decide how many servers how many servers, how many db are there? How many caches are there? Do we need more servers to read/write?

- ✓ DAU/MAU – daily active users/monthly active users (assumption in interview)
- ✓ Throughput – how many requests are in/out in system process each second
- ✓ Storage/Memory – what is the amount of storage that will be required.
- ✓ Network/Bandwidth – how much data goes in and out of system.

Step 3. API design

- ❖ Endpoint – door to specific part of your system
- ❖ Body – what do we want to send to this part of system
- ❖ Response – output we're getting

For eg. Youtube:

When uploading a video on youtube:

POST: /videos

“file”: “myvideo.mp4”

“title”: “How to design APIs”

“tags”: [“api”, “design”]

Response

{

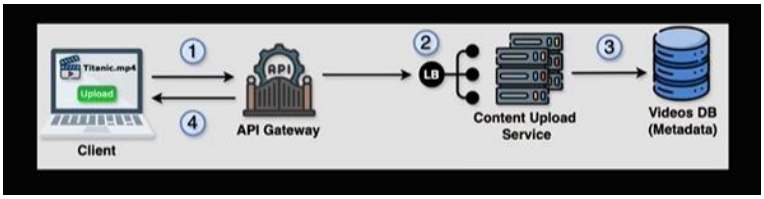
“videoId” :“1234”,

“status”: “success”

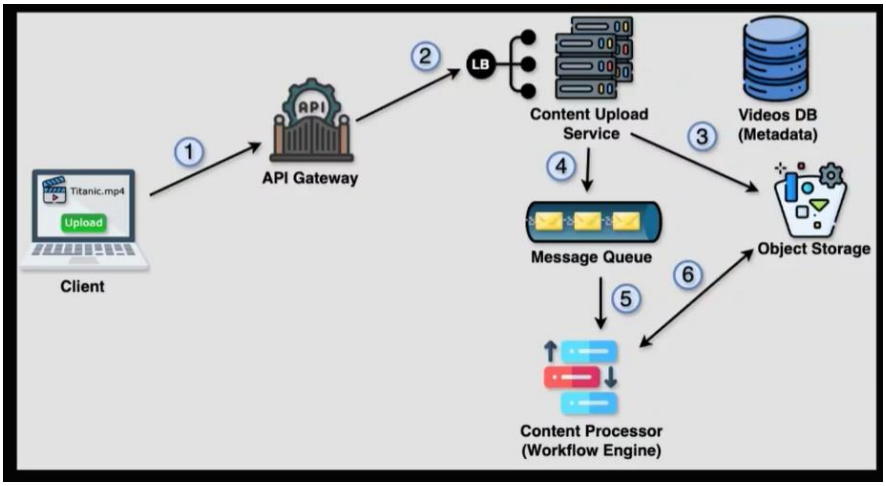
}

Step 4: High Level Design

In high level design part we start drawing box diagrams on white board or shared sheet to give high level overview of overall system.



As we do ahead we move to add more complexities and the process is gradual.



Step 4: Deep Dive

To deep dive into certain sections of the system, database selection, database modelling, caching.

- What is the data base type for storing video metadata?
- Video storage distribution using CDN?